PokeRush

Resumen del Proyecto - Archivos creados y modificados

Grupo 4 - ESTFA 2026 - Roles: Koyik (PM / DBA), Brunello (Developer), Andrusiani (Developer) - Lenguaje: C# - Motor: Unity (URP 2D) - Color del grupo: Violeta.

PokeRush es un juego 2D estilo Pokemon (las criaturas se llaman **Poken** y su vida son **PS**). El codigo sigue el patron **MVC** con capas de apoyo Services, Data y Core, y persistencia en SQLite. El repositorio esta organizado en cuatro carpetas: **Codigo**, **Diseno**, **Documentacion** y **Contexto IA**.

1. Planning

Tareas planificadas para este sprint, por rol:

UX

• Buscar sprites de NPC, Pokemons y fondos.

• Crear 3 pantallas: Inicio, Principal y HUD de batalla.

DEV

1. Cimientos del proyecto (Core). Definir los tipos base del juego: enums de tipos de Poken, tipos de ataque, fases de batalla, estados alterados, estados globales del juego y flags de historia. Es la base sobre la que se apoya todo lo demas.

2. Modelado de datos con ScriptableObjects (Data). Crear las 17 definiciones que describen el contenido del juego: especies de Poken, movimientos, items (curacion, combate, especiales, cosmeticos), personajes y su jerarquia (Character → NPC → Enemy/Boss/Nurse/Vendor), dialogos, eventos, tabla de XP y la configuracion del Boss Rush. Incluye disenar la herencia entre definiciones.

3. Logica de negocio (Models). Implementar el estado y las reglas en runtime: jugador, instancia de Poken (nivel/XP/PS), batalla por turnos, enemigos, inventario, movimientos, NPCs y progreso de historia. Es la capa sin Unity UI donde vive la logica real.

4. Interfaz de usuario (Views). Construir las vistas uGUI: HUD de batalla, barra de PS, vista del Poken, equipo, inventario, dialogos, cinematicas y menu de pausa.

5. Control y flujo de juego (Controllers). Conectar modelo y vista: movimiento del jugador por grilla, entrada con el nuevo Input System, orquestacion de la batalla, control del mapa, dialogos y eventos. (Es donde hubo trabajo reciente: refactor de PlayerController y PlayerInputController.)

6. Infraestructura (Managers). El andamiaje que sostiene todo: GameManager (singleton central), carga de escenas y audio.

DBA

Servicios (Services). El andamiaje que sostiene todo: persistencia en SQLite (DatabaseService + DatabaseRows), guardado/carga de partida y repositorio de definiciones.

PM

• Segundo Entregable.

• DFD Nivel 0 y Nivel 1.

2. Codigo (Codigo/PokeRush/Assets/Scripts)

Proyecto Unity en C# con arquitectura MVC. Cada script, su funcion y su responsable:

Core - enums y tipos base

Archivo	Funcion	Responsable
AttackType.cs	Enum con las categorias de ataque (fisico, especial, etc.).	Andrusiani
BattlePhase.cs	Enum con las fases de una batalla por turnos.	Andrusiani
GameState.cs	Enum con los estados globales del juego (mapa, batalla, menu...).	Brunello
PokenType.cs	Enum con los tipos elementales de los Poken.	Brunello
StatusEffect.cs	Enum de estados alterados (quemado, congelado, paralizado, etc.).	Andrusiani
StoryFlag.cs	Enum de banderas de progreso de la historia.	Brunello

Data - ScriptableObjects (definiciones)

Archivo	Funcion	Responsable
PokenDefinition.cs	Define una especie de Poken (stats base, tipo, sprites).	Brunello
MoveDefinition.cs	Define un movimiento/ataque y sus parametros.	Brunello
CharacterDefinition.cs	Definicion base de un personaje del juego.	Brunello
NPCDefinition.cs	Definicion de un NPC; hereda de CharacterDefinition.	Brunello
EnemyDefinition.cs	Definicion de un enemigo combatiente; hereda de NPCDefinition.	Brunello
BossDefinition.cs	Definicion de un jefe (boss); hereda de NPCDefinition.	Brunello
BossRushDefinition.cs	Configura la secuencia de jefes del modo Boss Rush.	Brunello
EnemyTeamEntry.cs	Una entrada del equipo enemigo: que especie usa y a que nivel.	Brunello
ExperienceTableDefinition.cs	Tabla de XP necesaria para subir de nivel.	Brunello
NurseDefinition.cs	Definicion de la enfermera que cura al equipo.	Andrusiani
VendorDefinition.cs	Definicion de un vendedor/tienda.	Andrusiani
DialogueDefinition.cs	Define un dialogo (lineas y hablantes).	Andrusiani
EventDefinition.cs	Define un evento disparable del mundo.	Andrusiani
BattleItemDefinition.cs	Define un objeto usable en combate.	Andrusiani
HealItemDefinition.cs	Define un objeto de curacion.	Andrusiani
SpecialItemDefinition.cs	Define un objeto especial/clave.	Andrusiani
CosmeticDefinition.cs	Define un cosmetico equipable en un Poken.	Andrusiani

Models - estado y logica de negocio

Archivo	Funcion	Responsable
PlayerModel.cs	Estado del jugador: equipo, dinero e inventario.	Brunello
PokenModel.cs	Instancia de un Poken: nivel, XP, PS y estado en juego.	Brunello
BattleModel.cs	Logica y estado de una batalla por turnos.	Brunello
EnemyModel.cs	Instancia de un enemigo con su equipo y recompensa.	Brunello
MoveModel.cs	Instancia de un movimiento aprendido por un Poken.	Brunello
InventoryModel.cs	Gestion del inventario de objetos del jugador.	Andrusiani

Archivo	Funcion	Responsable
NPCModel.cs	Estado base de un NPC en ejecucion.	Andrusiani
NurseModel.cs	Logica de la enfermera; hereda de NPCModel.	Andrusiani
VendorModel.cs	Logica del vendedor/tienda; hereda de NPCModel.	Andrusiani
StoryModel.cs	Estado del progreso de la historia (flags).	Andrusiani

Views - interfaz de usuario (uGUI)

Archivo	Funcion	Responsable
BattleHudView.cs	HUD de batalla: muestra el estado del combate.	Andrusiani
HealthBarView.cs	Barra de PS de un Poken.	Andrusiani
PokenView.cs	Render visual de un Poken en pantalla.	Andrusiani
TeamView.cs	Pantalla del equipo de Poken del jugador.	Andrusiani
InventoryView.cs	Pantalla del inventario de objetos.	Brunello
DialogueView.cs	Cuadro de dialogos.	Brunello
CinematicView.cs	Reproduce cinematicas/secuencias.	Brunello
PauseMenuView.cs	Menu de pausa.	Brunello

Controllers - control y entrada

Archivo	Funcion	Responsable
PlayerController.cs	Movimiento del jugador por grilla en el mapa. (modificado)	Brunello
PlayerInputController.cs	Entrada del nuevo Input System definida por codigo. (modificado)	Brunello
BattleController.cs	Orquesta el flujo de la batalla.	Brunello
MapController.cs	Controla el mapa y las transiciones.	Andrusiani
DialogueController.cs	Gestiona la reproduccion de dialogos.	Andrusiani
EventController.cs	Dispara y resuelve eventos del mundo.	Andrusiani

Managers - infraestructura

Archivo	Funcion	Responsable
GameManager.cs	Singleton central que coordina el juego, escenas y audio.	Brunello

Services - persistencia e infraestructura (DBA)

Archivo	Funcion	Responsable
DatabaseService.cs	Acceso a la base de datos SQLite.	Koyik
DatabaseRows.cs	Clases que mapean las filas de las tablas (Player, Poken...).	Koyik
SaveService.cs	Guardado y carga de la partida.	Koyik
DefinitionRepository.cs	Resuelve los ScriptableObjects por nombre/ID en runtime.	Koyik
SceneLoaderService.cs	Carga y cambio de escenas de Unity.	Koyik
AudioService.cs	Reproduccion de musica y efectos de sonido.	Koyik

Ademas se agrego la escena **Assets/Scenes/try.unity** (prueba de movimiento) y se removio Visual Scripting (no usado) para resolver el conflicto de SQLite y los errores de paquetes en Unity 6.

3. Documentacion

Documentacion General

Archivo	Funcion	Responsable
Arquitectura.md	Describe el motor, lenguaje y el patron MVC con sus capas.	Equipo (todos)
Historias_De_Usuario.md	Historias de usuario en formato INVEST con criterios de aceptacion.	Equipo (todos)
DFD/DFD_Nivel0_Contexto_PokeRush (.png/.svg)	Diagrama de flujo de datos de nivel 0 (contexto del sistema).	Equipo (todos)
DFD/DFD_Nivel1_PokeRush (.png/.svg)	Diagrama de flujo de datos de nivel 1 (procesos principales).	Equipo (todos)
Segundo Entregable Grupo X PokeRush.pdf	Documento del segundo entregable del proyecto.	Equipo (todos)

Documentacion del Codigo

Archivo	Funcion	Responsable
Como_Crear_Un_Enemigo.md	Guia paso a paso para dar de alta un enemigo o boss en Unity.	Brunello - Andrusiani

Seguimiento del sprint

Archivo	Funcion	Responsable
dailys-2026-06-25.xlsx	Registro de las dailys del sprint: integrante, rol, dia y tarea.	Equipo (todos)

Base de Datos (DBA)

Archivo	Funcion	Responsable
DER.png	Diagrama Entidad-Relacion de la base de datos.	Koyik
MER.pdf	Modelo Entidad-Relacion.	Koyik
MR.pdf	Modelo Relacional.	Koyik
Diccionario de Datos	Descripcion de cada tabla y atributo de la base de datos.	Koyik
Pokerush.sql	Script SQL con la creacion de las tablas.	Koyik
Pokerush	Base de datos SQLite del juego.	Koyik

4. Diseno (UX)

Archivo	Funcion	Responsable
Graphics/	Assets graficos varios (~17.800 archivos): sprites, animaciones, tilesets y efectos.	Equipo (todos)
Mockups/ (3 archivos)	Mockups de pantalla de inicio, menu principal y HUD de batalla.	Equipo (todos)
Pantallas/ (3 archivos)	Disenos finales de pantalla de inicio, menu principal y HUD de batalla.	Equipo (todos)

5. Contexto IA

Carpeta con el **grafo de conocimiento** del proyecto generado por graphify (graph.json, graph.html, GRAPH_REPORT.md, manifest y cache): mapea archivos, relaciones y estructura del codigo para ayudar a la

IA a navegarlo. Incluye tambien **tools/build_project_charter_docx.py**, un script que genera el Project Charter en formato Word a partir de una plantilla.